

Lab Equipment Booking & Maintenance Coordination

*Microservices Architecture — Individual Assignment 2026, Autumn
Technical Report*

2.1 Application Overview

Problem being addressed

University labs need equipment (oscilloscopes, 3D printers, microscopes, etc.) to be booked by students and researchers, while lab administrators need equipment availability to reflect real maintenance status rather than being tracked separately in a spreadsheet. This system connects the booking workflow to the equipment/maintenance record so that a faulty piece of equipment reported at the end of a booking automatically becomes unavailable for future bookings until it is repaired.

Responsibilities of each domain microservice

Booking Service (Service A) owns the booking workflow: creating, reading, completing, cancelling and deleting bookings, and issuing JWTs for authentication (register/login). It is the primary business workflow entry point for students.

Equipment Service (Service B) owns the equipment catalog and maintenance records: full CRUD on equipment, and maintenance record creation/status updates. It is the supporting resource Booking Service depends on.

Relationship between the domain entities

A Booking references an Equipment by ID. Before a booking is confirmed, Booking Service synchronously asks Equipment Service whether that equipment is AVAILABLE. When a booking is completed and the equipment is reported faulty, Booking Service asynchronously publishes a MaintenanceRequested event; Equipment Service consumes it, flips the equipment's status to UNDER_MAINTENANCE, and opens a new MaintenanceRecord referencing the equipment and the originating booking.

Rationale for the chosen domain

This domain was chosen over a generic orders/products system because it has a genuine, asymmetric two-service relationship with a natural synchronous dependency (booking must check current equipment status) and a natural asynchronous side effect (maintenance

reporting), rather than two services that could just as easily be one. It also gave a believable reason for two different role-based authorisation patterns (students book; only lab admins manage the equipment catalog and delete bookings).

Confirmation that the domain differs from previous group work

[Insert one sentence here identifying your previous group project's domain, to confirm this is different, e.g.: "Our group project was an e-commerce order/inventory system; this individual assignment uses an unrelated university lab equipment domain."]

2.2 Architecture Overview

The diagram below shows all required components: the API Gateway as the sole client entry point, Eureka for service discovery, the Config Server for externalised configuration, the two domain microservices with their dedicated databases, RabbitMQ as the messaging infrastructure, and Zipkin as the tracing infrastructure. Solid black arrows are synchronous REST calls; dashed orange arrows are the asynchronous RabbitMQ event; thin dotted grey lines represent the cross-cutting infrastructure concerns (service registration, config fetch, trace export) common to all three application services.

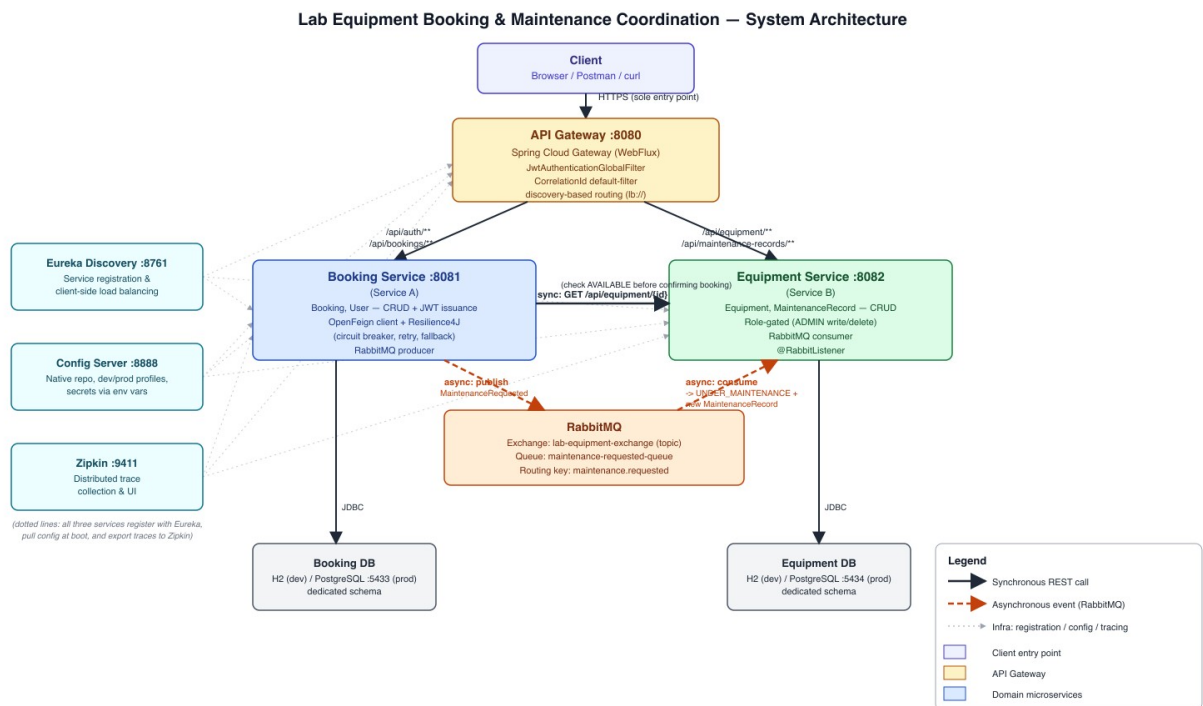


Figure 1: System architecture, synchronous and asynchronous communication paths

Responsibilities of each component

Component	Responsibility
API Gateway (:8080)	Sole client entry point. Discovery-based routing to booking-service and equipment-service. Enforces JWT authentication via a global filter before forwarding any request. Applies a CorrelationId filter to every route.
Eureka Discovery Server (:8761)	Service registry. All three application services (Gateway, Booking, Equipment) register on startup and are resolved via client-side load balancing (lb://) rather than hardcoded hosts/ports.
Config Server (:8888)	Serves externalised configuration (native file-based repo) to every client, with dev and prod profiles per service. Sensitive values (JWT secret, DB passwords) are environment-variable placeholders, never hardcoded.
Booking Service (:8081)	Owns Booking and User entities. Issues JWTs. Synchronously calls Equipment Service (OpenFeign + Resilience4J) to check availability. Publishes the MaintenanceRequested event to RabbitMQ.
Equipment Service (:8082)	Owns Equipment and MaintenanceRecord entities. Consumes the MaintenanceRequested event and updates its own state accordingly.
RabbitMQ	Messaging infrastructure. Topic exchange lab-equipment-exchange, queue maintenance-requested-queue, routing key maintenance.requested.
Zipkin (:9411)	Collects and visualises distributed traces exported via Micrometer Tracing + OpenTelemetry from all three application services.
Booking DB / Equipment DB	Dedicated database per service (H2 in-memory for the dev profile, separate PostgreSQL instances on :5433/:5434 for the prod profile) - no shared schema between services.

Significant architectural choices

Three significant choices are documented as full ADRs in Section 2.4: enforcing authentication centrally at the Gateway rather than in each service (ADR-001), using asynchronous messaging for the maintenance-reporting side effect rather than a second synchronous call (ADR-002), and

wrapping the synchronous Booking→Equipment call with Resilience4J circuit breaker/retry rather than leaving it unprotected (ADR-003).

2.3 Implementation Evidence

All evidence below was captured live against the running system (all five services, RabbitMQ, and Zipkin, started per the README) rather than reconstructed after the fact. Raw curl transcripts are used for REST evidence (request URL, method, payload, response body, and status code are all directly visible and precise) alongside dashboard screenshots for infrastructure state.

REST APIs — full CRUD lifecycle

The following transcript demonstrates CREATE, READ, UPDATE, DELETE, a validation error, and the resulting 404 after deletion, all against /api/equipment through the Gateway. Corresponding screencast timestamp: MM:SS-MM:SS (TBD).

```
=== 5. GET /api/equipment/4 (READ) -> 200 ===
HTTP/1.1 200 OK
X-Correlation-Id: 0064a03a-2956-4133-b025-f59d0936c177

{"id":4,"name":"3D Printer","category":"Fabrication","location":"Lab C", "status":"AVAILABLE", "conditionNotes":"New", "createdAt":"2026-07-30T17:42:25.725888Z", "updatedAt":"2026-07-30T17:42:25.725888Z"}

=== 6. PUT /api/equipment/4 as ADMIN (UPDATE) -> 200 ===
HTTP/1.1 200 OK
X-Correlation-Id: 33d8fd99-81ad-4ece-a8e6-fd153ec1cbab

{"id":4,"name":"3D Printer","category":"Fabrication","location":"Lab C", "status":"UNDER_MAINTENANCE", "conditionNotes":"Nozzle jammed", "createdAt":"2026-07-30T17:42:25.725888Z", "updatedAt":"2026-07-30T17:42:42.452306Z"}

=== 7. POST /api/equipment with INVALID body (missing required fields) -> 400
Validation ===
HTTP/1.1 400 Bad Request
X-Correlation-Id: b351867d-cfc9-45f5-9fde-86d9d528aaf3

{"timestamp":"2026-07-30T17:42:42.681224Z","status":400,"error":"Validation Failed","fieldErrors":{"name":"name is required","status":"status is required","category":"category is required","location":"location is required"}}

=== 8. DELETE /api/equipment/4 as ADMIN -> 204 No Content (DELETE) ===
HTTP/1.1 204 No Content
X-Correlation-Id: 7de2ca76-6ba2-4c65-90f6-53ec291dc53b

=== 9. GET /api/equipment/4 after delete -> 404 Not Found ===
HTTP/1.1 404 Not Found
X-Correlation-Id: ee53a1fd-393b-4362-9dbc-1165883acdb9

{"timestamp":"2026-07-30T17:42:43.827709Z","status":404,"error":"Equipment 4 not found"}
```

Gateway, Service Discovery and Configuration

All three application services (API-GATEWAY, BOOKING-SERVICE, EQUIPMENT-SERVICE) are shown below registered and UP in the Eureka dashboard. Corresponding screencast timestamp: MM:SS-MM:SS (TBD).

Renews threshold	6
Renews (last min)	6

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - localhost:api-gateway:8080
BOOKING-SERVICE	n/a (1)	(1)	UP (1) - localhost:booking-service:8081
EQUIPMENT-SERVICE	n/a (1)	(1)	UP (1) - localhost:equipment-service:8082

General Info

Name	Value
total-avail-memory	103mb
num-of-cpus	8
current-memory-usage	45mb (43%)
server-uptime	02:16
registered-replicas	

Figure 2: Eureka dashboard — all three services registered and UP

Configuration is fetched from the Config Server at boot; the dev profile serves H2 datasource settings and DEBUG-level logging, the prod profile serves PostgreSQL settings sourced from environment variables and WARN-level logging (see [config-server/src/main/resources/config/booking-service-dev.yml](#) vs [booking-service-prod.yml](#)).

Security

The transcript below shows a successful login issuing a JWT, an unauthenticated request rejected with 401 directly by the Gateway (before reaching any domain service), and a request rejected with 403 for a correctly-authenticated USER attempting an ADMIN-only write.

Corresponding screencast timestamp: MM:SS-MM:SS (TBD).

```
=== 1. POST /api/auth/login (admin) - 200 ===
HTTP/1.1 200 OK
X-Correlation-Id: c5e97bdd-7175-4b5f-953c-192fdf3d4452

{"token":"eyJhbGciOiJIUzM4NCJ9...
(truncated)...CLR7jgxPodCSlnuNTz2Zqwt","username":"admin","role":"ADMIN","expiresI
nMs":3600000}

=== 2. GET /api/equipment with NO token -> 401 ===
HTTP/1.1 401 Unauthorized
X-Auth-Error: Missing bearer token
```

```

=== 3. POST /api/equipment as STUDENT (wrong role) -> 403 ===
HTTP/1.1 403 Forbidden
X-Correlation-Id: 5e85ea9c-54eb-4011-8bdf-ca81e0154cd2

{"timestamp":"2026-07-30T17:42:25.643544Z","status":403,"error":"Requires role:
ADMIN"}

=== 4. POST /api/equipment as ADMIN -> 201 Created (CREATE) ===
HTTP/1.1 201 Created
X-Correlation-Id: 7aec9f0a-c210-445d-80a8-e346fd0d6f74
Location: /api/equipment/4

{"id":4,"name":"3D Printer","category":"Fabrication","location":"Lab
C","status":"AVAILABLE","conditionNotes":"New","createdAt":"2026-07-
30T17:42:25.725888Z","updatedAt":"2026-07-30T17:42:25.725888Z"}

```

Service-to-Service Communication and Resilience

Equipment Service was stopped to simulate an outage. The circuit breaker's state was queried before and after repeated failing calls, showing the transition CLOSED → OPEN (actual failure rate 80%, exceeding the 50% threshold) → automatic HALF_OPEN after the configured wait duration. While OPEN, calls fail immediately without attempting the network call (notPermittedCalls), and the caller receives a clear 503 in roughly a second rather than a hang. Corresponding screencast timestamp: MM:SS-MM:SS (TBD).

```

=== Booking Service circuit breaker state BEFORE failure (CLOSED) ===
{
  "details": {
    "equipmentService": {
      "details": {
        "failureRate": "-1.0%",
        "failureRateThreshold": "50.0%",
        "slowCallRate": "-1.0%",
        "slowCallRateThreshold": "100.0%",
        "bufferedCalls": 1,
        "slowCalls": 0,
        "slowFailedCalls": 0,
        "failedCalls": 0,
        "notPermittedCalls": 0,
        "state": "CLOSED"
      },
      "status": "UP"
    }
  },
  "status": "UP"
}

=== Killing equipment-service to simulate an outage ===
=== POST /api/bookings while Equipment Service is DOWN ===
HTTP/1.1 503 Service Unavailable
X-Correlation-Id: 291756ba-ba0c-41fa-bf9b-78a5d65eae3a

{"timestamp":"2026-07-30T17:43:14.309783Z","status":503,"error":"Equipment Service
is currently unavailable - could not verify equipment 1. Please try again
shortly."}

```

```

(request completed in ~1.3s total - fast, clear failure rather than a hang)

=== 6 rapid attempts while Equipment Service down -> all 503, circuit breaker
trips OPEN ===
attempt 1: 503
attempt 2: 503
attempt 3: 503
attempt 4: 503
attempt 5: 503
attempt 6: 503

=== Circuit breaker state AFTER repeated failures (OPEN - fails fast, no network
call attempted) ===
{
  "details": {
    "equipmentService": {
      "details": {
        "failureRate": "80.0%",
        "failureRateThreshold": "50.0%",
        "slowCallRate": "0.0%",
        "slowCallRateThreshold": "100.0%",
        "bufferedCalls": 5,
        "slowCalls": 0,
        "slowFailedCalls": 0,
        "failedCalls": 4,
        "notPermittedCalls": 17,
        "state": "OPEN"
      },
      "status": "CIRCUIT_OPEN"
    }
  },
  "status": "UNKNOWN"
}

```

Note: failureRateThreshold (50%) was exceeded (80% actual), so the breaker opened after the minimum-number-of-calls (5) threshold was reached. notPermittedCalls=17 shows the breaker then started rejecting calls immediately (no network attempt) while OPEN. After wait-duration-in-open-state (10s), it automatically transitioned to HALF_OPEN to test recovery - confirmed by a follow-up query:

```

{
  "details": {
    "equipmentService": {
      "details": {
        "state": "HALF_OPEN",
        "status": "CIRCUIT_HALF_OPEN"
      }
    }
  }
}

```


Asynchronous Messaging

The RabbitMQ management UI below shows the lab-equipment-exchange (topic exchange) and maintenance-requested-queue with live message rates, confirming both are declared correctly and processing traffic. Corresponding screencast timestamp: MM:SS-MM:SS (TBD).

The screenshot shows the RabbitMQ Management UI with the 'Exchanges' tab selected. The interface includes a top navigation bar with 'Overview', 'Connections', 'Channels', 'Exchanges', 'Queues and Streams', and 'Admin'. The 'Exchanges' tab displays a table of all exchanges. The 'lab-equipment-exchange' is highlighted, showing it is a topic exchange with a message rate of 0.00/s.

Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
/	(AMQP default)	direct	D			
/	amq.direct	direct	D			
/	amq.fanout	fanout	D			
/	amq.headers	headers	D			
/	amq.match	headers	D			
/	amq.rabbitmq.log	topic	D I			
/	amq.rabbitmq.trace	topic	D I			
/	amq.topic	topic	D			
/	lab-equipment-exchange	topic	D	0.00/s	0.00/s	

Figure 3: RabbitMQ Exchanges — lab-equipment-exchange with active message rate

The screenshot shows the RabbitMQ Management UI with the 'Queues and Streams' tab selected. The interface includes a top navigation bar with 'Overview', 'Connections', 'Channels', 'Exchanges', 'Queues and Streams', and 'Admin'. The 'Queues and Streams' tab displays a table of all queues. The 'maintenance-requested-queue' is highlighted, showing it is a classic queue with a message rate of 0.00/s.

Virtual host	Name	Type	Features	State	Ready	Unacked	Total	Message rates	+/-
								incoming deliver / get ack	
/	maintenance-requested-queue	classic	D Args	running	0	0	0	0.00/s 0.00/s 0.00/s	

Figure 4: RabbitMQ Queues — maintenance-requested-queue

See ADR-002 for the justification of using asynchronous messaging rather than a synchronous call for this specific interaction.

Observability and Distributed Tracing

The trace below shows the full synchronous chain for booking creation in one trace: the Gateway receives the request, routes to Booking Service, which in turn calls Equipment Service via OpenFeign to check availability — all three services appear under a single trace ID. Corresponding screencast timestamp: MM:SS-MM:SS (TBD).

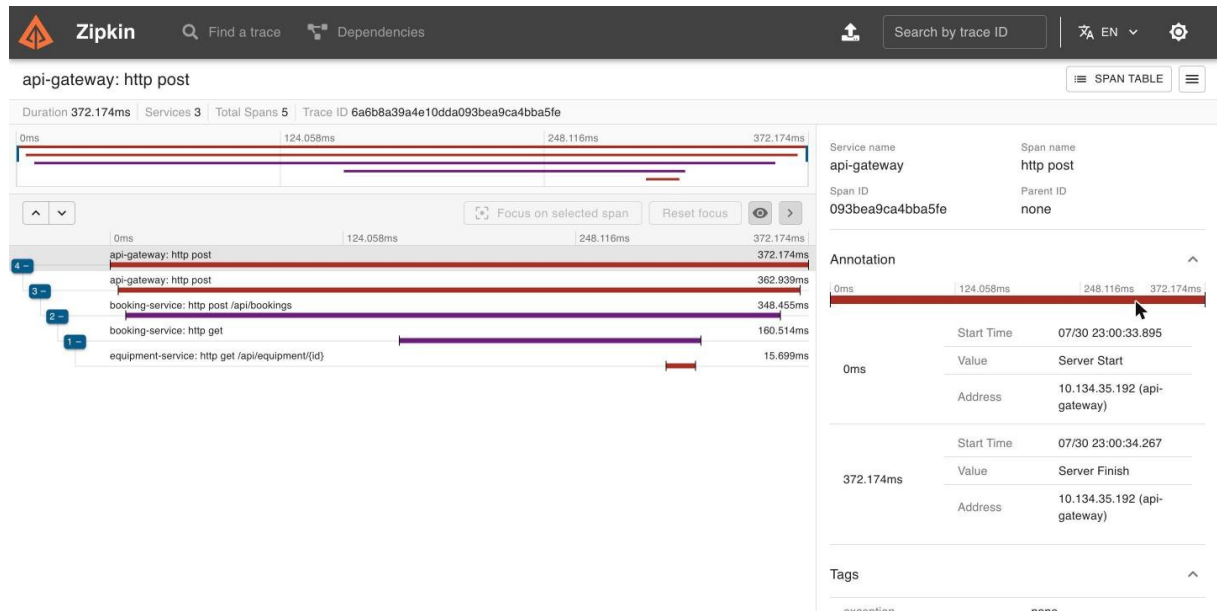


Figure 5: Zipkin trace — Gateway to Booking Service to Equipment Service (sync)

The trace below shows propagation through the asynchronous path: Booking Service's publish to lab-equipment-exchange/maintenance.requested and Equipment Service's receive from maintenance-requested-queue both appear as spans under the same trace ID as the originating booking-completion request, demonstrating trace propagation across the message broker.

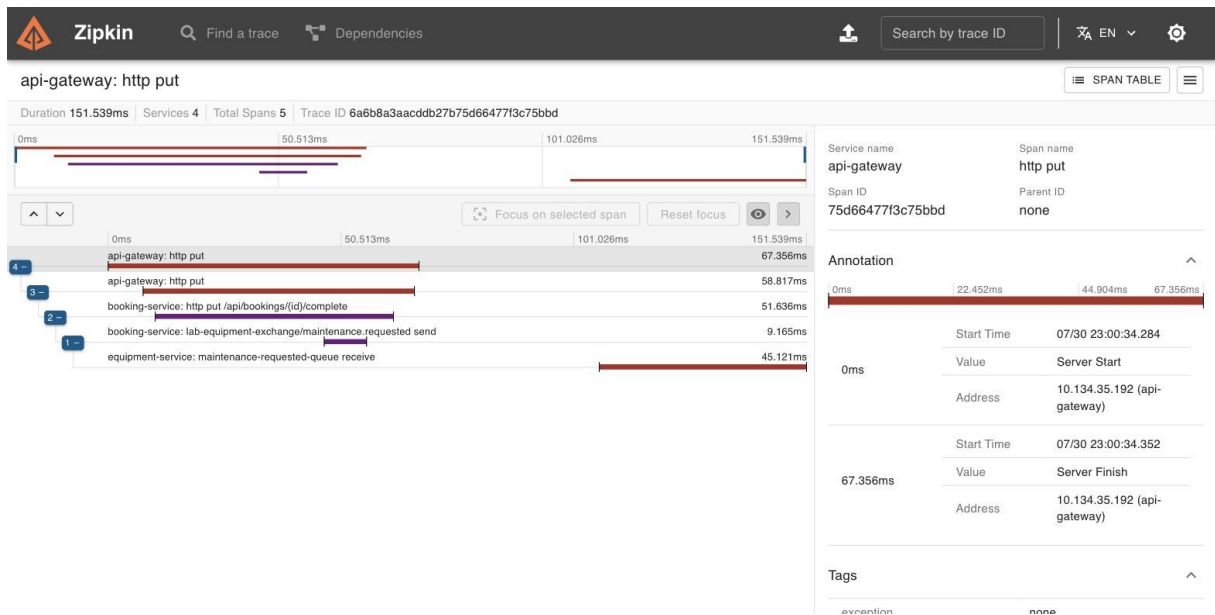


Figure 6: Zipkin trace — async publish/consume spans under one trace ID

Correlation IDs (independent of the OTel trace ID, added by the Gateway's CorrelationId filter) and trace/span IDs both appear in every log line, e.g.: [booking-service,6a6b72894094e59bb5fa4d2ddf162e91,5971245556703769] — see logs/*.log for full output during the screencast recording.

2.4 Architecture Decision Records (ADRs)

Full ADRs (Context, Decision, Alternatives Considered, Consequences, Implementation Evidence) are included in Appendix A and as standalone files in docs/adr/. Summaries follow.

ADR-001: Gateway-Centred Security

Decision: authentication is enforced once, centrally, at the API Gateway (JwtAuthenticationGlobalFilter), which validates the JWT and forwards the identity via X-Auth-User/X-Auth-Roles headers; each service additionally runs a lightweight RoleGuard so a request that bypasses the Gateway is still rejected rather than trusted implicitly. Rejected alternatives: per-service Spring Security + JWT decoding (duplicates verification logic across services); trusting downstream services completely with no re-check (defeats the "avoid direct access" requirement); OAuth2 with an external Authorization Server (disproportionate operational cost for two roles). See Appendix A.1 for full rationale and consequences.

ADR-002: Event-Driven Communication for Maintenance Reporting

Decision: booking completion publishes a MaintenanceRequested event to RabbitMQ rather than calling Equipment Service synchronously, because the caller does not need the result, the event must survive Equipment Service being temporarily down, and this interaction's failure domain should be decoupled from booking completion's success. Rejected alternatives: a synchronous call mirroring the availability check (couples booking completion to Equipment Service's availability for a side effect the caller doesn't wait on); Kafka (unneeded complexity for one event type at this scale); Spring Cloud Stream (adds an abstraction layer not needed for one exchange/queue). See Appendix A.2 for full rationale and consequences.

ADR-003: Resilience and Failure Handling for the Sync Booking to Equipment Call

Decision: the synchronous availability check is wrapped with Resilience4J Retry (3 attempts, 500ms apart) and CircuitBreaker (opens at 50%+ failure rate over a 10-call window, min. 5 calls), backed by 2s Feign connect/read timeouts, with a fallback that returns a clear 503 instead of a hang or raw exception. Rejected alternatives: no resilience wrapper (violates the requirement outright); retry only (keeps hammering a genuinely down dependency); circuit breaker only (would trip on single transient blips retry would otherwise absorb); async TimeLimiter (would force the synchronous booking flow onto an async model for no real benefit over Feign's own timeout). See Appendix A.3 for full rationale and consequences.

2.5 Screenshot Cross-Reference Table (Mandatory)

Screenshot timestamps below are marked TBD - to be filled in once the screenshot is recorded.

All other columns are already accurate to this report's structure.

Requirement	Section	Screenshot	ADR
REST CRUD (Equipment): create/read/update/delete, status codes, validation	2.3 REST APIs	TBD	-
Eureka registration (all 3 services UP)	2.3 Gateway/Discovery/Config	TBD	-
Config Server: dev/prod profiles, externalised secrets	2.3 Gateway/Discovery/Config	TBD	-
Gateway routing + CorrelationId filter	2.2 / 2.3	TBD	ADR-001
JWT login, 401 unauth., 403 wrong role	2.3 Security	TBD	ADR-001
Sync call Booking → Equipment (OpenFeign)	2.3 Resilience	TBD	ADR-003
Resilience4J: circuit open, fallback 503	2.3 Resilience	TBD	ADR-003
Async event publish/consume (RabbitMQ)	2.3 Messaging	TBD	ADR-002
State change: UNDER_MAINTENANCE + record	2.3 Messaging	TBD	ADR-002
Trace: sync chain (Gateway/Booking/Equip.)	2.3 Observability	TBD	-
Trace: async chain (publish/consume spans)	2.3 Observability	TBD	ADR-002
Correlation IDs in logs	2.3 Observability	TBD	-

3. Source Code Repository

Repository link: [INSERT GIT REMOTE URL HERE - push the local repository at lab-equipment-microservices/ to GitHub/GitLab and share access with the module team].

The repository contains a clear README with build/run instructions, and demonstrates incremental development through multiple meaningful commits (parent scaffold; discovery + config server; gateway; equipment service; booking service; docker-compose + smoke test; README) rather than a single bulk upload.

Appendix A: Full Architecture Decision Records

A.1 ADR-001: Gateway-Centred Security

Status

Accepted

Context

The system exposes two domain microservices (Booking Service, Equipment Service) behind a Spring Cloud Gateway. The assignment requires JWT-based authentication, enforced before requests reach a downstream microservice, plus role-based authorisation with at least two differently protected endpoint categories (read-only for any authenticated user; write/delete restricted to ADMIN).

A decision was needed on **where** authentication is enforced:

1. At the Gateway only, with downstream services trusting a forwarded identity header.
2. At each downstream microservice independently (e.g. Spring Security + JWT decoding in every service).
3. Duplicated in both places.

The domain has two roles only (`USER`, `ADMIN`), issued by Booking Service's `/api/auth/login` and `/api/auth/register` endpoints (`AuthController`, `JwtTokenProvider`), signed with a shared HMAC secret sourced from Config Server (`jwt.secret`, environment-variable backed, never hardcoded).

Decision

Authentication is enforced centrally at the API Gateway via `JwtAuthenticationGlobalFilter` (`api-gateway/src/main/java/com/labequip/gateway/security/JwtAuthenticationGlobalFilter.java`):

- Every request not matching `security.public-paths` (`/api/auth/\*\*`, `/actuator/\*\*`) must present

a valid `Bearer` JWT, verified via `JwtValidator` against the shared secret.

- On success, the Gateway enriches the request with `X-Auth-User` and `X-Auth-Roles` headers before

forwarding it downstream (a form of trusted-header propagation, not re-issuing a token).

- On failure (missing/invalid/expired token), the Gateway returns `401` immediately - the request

never reaches Booking Service or Equipment Service.

Each downstream service additionally runs a lightweight `RoleGuard` (`equipment-service/.../security/RoleGuard.java`, `booking-service/.../security/RoleGuard.java`) that re-checks `X-Auth-User`/`X-Auth-Roles` before performing the operation:

- `EquipmentController`: `GET` requires any authenticated user; `POST`/`PUT`/`DELETE` require `ADMIN`.
- `BookingController`: `GET`/`POST`/`complete`/`cancel` require any authenticated user; `DELETE`

requires `ADMIN`.

This is defence-in-depth, not duplicated authentication: the services trust the headers as an **identity assertion from a trusted network hop** (the Gateway), not as a token they independently verify. If a request reaches a service without those headers - i.e. it bypassed the Gateway - `RoleGuard.requireAuthenticated` rejects it with `401`, which also satisfies the requirement that "direct browser/client access to microservices should be avoided" by making direct access non-functional for anything but public health checks.

Alternatives Considered

1. ****Per-service Spring Security + JWT decoding**** - each service independently validates the JWT and applies `@PreAuthorize`. Rejected as the primary mechanism: it duplicates JWT parsing logic across every service, means a change to the auth scheme (e.g. rotating to OAuth2) has to be made N times, and does not stop a request that reaches a service directly (bypassing the Gateway) unless each service also re-implements the check - at which point the Gateway's filter is redundant work. Full Spring Security was also heavier than needed for two roles.
2. ****Trust downstream services completely (no RoleGuard)**** - simplest, but means any request that reaches a service directly (e.g. hitting `localhost:8082` instead of the Gateway) would be treated as authenticated with no role, defeating the "avoid direct access" requirement. Rejected.
3. ****OAuth2 with an external Authorization Server**** (Keycloak / Spring Authorization Server) - more standard for production and would support token introspection/refresh, but adds an extra moving part and setup/operational cost disproportionate to a two-role, coursework-scoped system. JWT issued directly by Booking Service was judged sufficient and simpler to demonstrate end

to

end within the assignment's scope.

Consequences

****Positive****

- Single point of authentication enforcement; adding a third microservice only requires adding a

route + optional `RoleGuard`, not re-implementing JWT verification.

- Clean separation: Gateway answers "is this request authenticated, and as whom", services answer

"is this identity allowed to do this specific operation".

- Satisfies the "at least two differently protected endpoint categories" requirement uniformly across both services (read vs. write/delete).

****Negative / trade-offs****

- The shared JWT secret must be identical across Gateway, Booking Service, and Equipment Service

(currently distributed via Config Server's shared `application.yml`) - a key rotation requires updating one place, but restarting all three processes.

- The `X-Auth-User`/`X-Auth-Roles` header trust model assumes the network between Gateway and the

domain services is not itself hostile (no mutual TLS or signed headers) - acceptable for this assignment's scope but would need hardening (e.g. mTLS or a short-lived signed hop token) in a

real production deployment.

- Feign calls from Booking Service to Equipment Service must manually forward these headers

(`FeignHeaderForwardingConfig`), since that hop also bypasses the Gateway.

Implementation Evidence

- `api-gateway/src/main/java/com/labequip/gateway/security/JwtAuthenticationGlobalFilter.java`
- `api-gateway/src/main/java/com/labequip/gateway/security/JwtValidator.java`
- `booking-service/src/main/java/com/labequip/booking/web/AuthController.java`
- `booking-service/src/main/java/com/labequip/booking/security/JwtTokenProvider.java`
- `equipment-service/src/main/java/com/labequip/equipment/security/RoleGuard.java`

- ``booking-service/src/main/java/com/labequip/booking/config/FeignHeaderForwardingConfig.java``
- Config: ``config-server/src/main/resources/config/application.yml`` (``jwt.secret``),

``api-gateway.yml`` (``security.public-paths``)

- Report evidence: Section 2.3 "Security" (``docs/evidence-crud-security.txt``, items 1-3: 200 login,

401 no-token, 403 wrong-role)

- Screencast timestamp: `**TBD - insert after recording**`

A.2 ADR-002: Event-Driven Communication for Maintenance Reporting

Status

Accepted

Context

When a student completes a booking and reports the equipment as faulty, two things must happen:

Booking Service must record the completion, and Equipment Service must be told to take the equipment out of service and open a maintenance record. The assignment requires at least one interaction in the system to use asynchronous messaging rather than a synchronous REST call, with

a justification for why messaging was the better fit here.

The alternative - having `BookingService.complete()` call Equipment Service synchronously (the same

way it already does for the availability check in `EquipmentAvailabilityService`) - was considered and rejected for this specific interaction.

Decision

`BookingService.complete()` (`booking-service/.../service/BookingService.java`) publishes a `MaintenanceRequestedEvent` to RabbitMQ (`MaintenanceEventPublisher`) after marking the booking

`COMPLETED`, instead of calling Equipment Service directly:

- Exchange: `lab-equipment-exchange` (topic), routing key `maintenance.requested`

(`booking-service/.../messaging/RabbitMQConfig.java`)

- Queue: `maintenance-requested-queue`, bound in Equipment Service

(`equipment-service/.../messaging/RabbitMQConfig.java`)

- Consumer: `MaintenanceRequestedListener.onMaintenanceRequested()` calls

`MaintenanceRecordService.handleMaintenanceRequested()`, which sets the equipment's status to

`UNDER_MAINTENANCE` and creates a new `MaintenanceRecord` - a genuine, meaningful state change, not a no-op notification.

- The event class `com.labequip.events.MaintenanceRequestedEvent` is deliberately duplicated

(same package + class name) in both services rather than shared as a library, so the two services remain independently deployable and the event is treated as a payload *contract*,

not

a compile-time dependency.

Why Asynchronous Rather Than Synchronous, For This Interaction Specifically

This is the interaction where async is *more* appropriate than the sync call used elsewhere in the

system (Booking → Equipment availability check), for reasons specific to this case:

1. *The caller does not need the result.* Confirming a booking genuinely depends on knowing whether the equipment is available *right now* - that's a natural synchronous read.

Completing

a booking does not depend on Equipment Service having already processed the maintenance flag;

the booking is complete regardless of whether Equipment Service is up, slow, or momentarily unreachable.

2. *Availability under partial outage.* If Equipment Service is down, a synchronous design would

force a choice between failing the booking-completion request (losing the user's action) or silently dropping the maintenance report. With messaging, RabbitMQ durably holds the event until Equipment Service is available again to consume it - the booking completes immediately either way, and the maintenance side effect is never lost.

3. *Decoupled failure domains.* A slow or failing Equipment Service should not make booking completion slow or failing. The sync call (availability check) *should* propagate Equipment Service's health into the booking flow (you cannot confirm a booking you can't verify) - this one deliberately should not.

4. *Natural one-to-many extension point.* A "maintenance requested" event is the kind of fact

other future consumers might care about (e.g. a notification service, an analytics service) -

a queue/exchange scales to multiple consumers without Booking Service knowing about them;

a

direct REST call would not.

Alternatives Considered

1. *Synchronous REST call from Booking Service to Equipment Service on completion* (mirroring the

existing Feign-based availability check). Rejected: couples booking completion's success/latency

to Equipment Service's availability for a side effect the caller doesn't need to wait on, and the assignment specifically asks for one interaction to demonstrate messaging instead.

2. *Kafka instead of RabbitMQ.* Considered, but RabbitMQ was chosen for this single point-to-point/topic-routed event: no need for Kafka's log-based replay, partitioning, or consumer-group semantics at this scale, and RabbitMQ's lower operational overhead (single container, simpler `docker-compose` entry, quicker to demonstrate) was a better fit for a

two-service, single-event-type system.

3. **Spring Cloud Stream abstraction over RabbitMQ** instead of raw `spring-boot-starter-amqp` +

`@RabbitListener`. Rejected for this assignment: Cloud Stream would add a binder abstraction layer that isn't needed for a single exchange/queue and would obscure the exchange/routing-key/queue wiring the report needs to evidence directly.

Consequences

Positive

- Booking completion is fast and does not depend on Equipment Service's availability.
- Equipment Service processes maintenance events at its own pace, and RabbitMQ durably queues them

if Equipment Service is temporarily down (queue is declared `durable: true`).

- New consumers of `MaintenanceRequested` can be added without touching Booking Service.

Negative / trade-offs

- Eventual consistency: there is a (normally sub-second) window where a booking is `COMPLETED` but

the equipment hasn't yet flipped to `UNDER_MAINTENANCE`. Acceptable for this domain (nobody is blocked waiting on it).

- No compensating transaction/outbox pattern was implemented: if `RabbitTemplate.convertAndSend`

fails synchronously (e.g. broker unreachable at publish time) after the booking row is already committed, the event is lost. For this assignment's scope this was accepted as a known simplification rather than implementing a transactional outbox.

- Cross-service type mapping required care: the default `DefaultClassMapper` in

`Jackson2JsonMessageConverter` resolves `__TypeId__` by fully-qualified class name, so both services' event classes must share the same package + class name (documented in both `RabbitMQConfig` classes) - a subtlety worth calling out since a mismatch fails silently at consume time with a `MessageConversionException`.

Implementation Evidence

- `booking-service/src/main/java/com/labequip/booking/messaging/MaintenanceEventPublisher.java`
- `booking-service/src/main/java/com/labequip/booking/messaging/RabbitMQConfig.java`

- ``equipment-service/src/main/java/com/labequip/equipment/messaging/MaintenanceRequestedListener.java``
- ``equipment-service/src/main/java/com/labequip/equipment/messaging/RabbitMQConfig.java``
- ``equipment-service/src/main/java/com/labequip/equipment/service/MaintenanceRecordService.java``

`(`handleMaintenanceRequested`)`

- RabbitMQ management UI evidence: ``docs/screenshots/02-rabbitmq-queue.jpg``,
``docs/screenshots/03-rabbitmq-exchange.jpg``
- Distributed trace showing the publish/consume hop: ``docs/screenshots/05-zipkin-async-trace.jpg``

`(spans: `booking-service: lab-equipment-exchange/maintenance.requested send` →
`equipment-service: maintenance-requested-queue receive`)`

- Report evidence: Section 2.3 "Asynchronous Messaging"
- Screencast timestamp: `**TBD - insert after recording**`

A.3 ADR-003: Resilience and Failure Handling for the Sync Booking → Equipment Call

Status

Accepted

Context

Booking Service must synchronously confirm equipment is `AVAILABLE` before confirming a booking

(the assignment's required "Service A must synchronously query Service B before completing at least one operation"). This call crosses a process and network boundary via Eureka + OpenFeign (`EquipmentClient`), so it can fail in ways a plain in-process method call cannot: Equipment Service can be down, slow, or network-partitioned. Without protection, a single slow/failing Equipment Service would make every booking-creation request hang or fail with a raw connection

exception, and repeated attempts would keep retrying against a service that isn't coming back soon - wasting threads/connections and degrading Booking Service itself.

Decision

The call is wrapped in `EquipmentAvailabilityService.fetchEquipment()`
(`booking-service/.../service/EquipmentAvailabilityService.java`) with two Resilience4J annotations, configured per-instance as `equipmentService` in Config Server
(`config-server/.../config/booking-service.yml`):

```
resilience4j:
  circuitbreaker:
    instances:
      equipmentService:
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true
  retry:
    instances:
      equipmentService:
        max-attempts: 3
        wait-duration: 500ms
  feign:
    client:
      config:
        equipment-service:
          connect-timeout: 2000
          read-timeout: 2000
```

- `**@Retry**` absorbs transient blips (e.g. a single dropped connection) with 3 attempts, 500ms

apart - short enough not to make the caller wait unreasonably long.

- `**Feign connect/read timeouts (2s)**` bound how long a single attempt can hang, so a slow (not

down) Equipment Service can't tie up a Booking Service thread indefinitely.

- `**`@CircuitBreaker`**` tracks the last 10 calls; once at least 5 calls have been made and 50% +

failed, it opens for 10 seconds, during which calls fail immediately (``notPermittedCalls``, verified in testing: see Implementation Evidence) without attempting the network call at all - protecting Booking Service from wasting resources on a service known to be down. It then half-opens automatically to test recovery with a limited number of trial calls.

- `**`fallbackMethod = "equipmentUnavailableFallback"`**` runs whenever the retries are exhausted or

the circuit is open, and throws a ``ResponseStatusException(503, ...)`` with a clear message - turning an opaque connection failure into a well-formed API response the client can act on, rather than a hung request or a raw 500.

Alternatives Considered

1. `**No resilience wrapper - let the Feign call fail/hang naturally.**` Rejected outright: violates the assignment's explicit resilience requirement and would mean any Equipment Service blip directly produces hung requests or ugly 500s at the Gateway.
2. `**Retry only, no circuit breaker.**` Would absorb transient failures but keeps hammering a genuinely-down dependency on every subsequent request, each still paying the full retry+timeout cost. Rejected: doesn't protect Booking Service's own resources under a sustained outage, which is the more realistic multi-minute failure mode a circuit breaker specifically addresses.
3. `**Circuit breaker only, no retry.**` Rejected: without a retry, a single transient TCP hiccup (not indicative of Equipment Service actually being down) would count as a hard failure and contribute to tripping the breaker unnecessarily; retry lets genuinely transient failures self-heal before they're counted against the breaker's failure budget.
4. `**`@TimeLimiter`` (async, ``CompletableFuture``-based) in addition to circuit breaker/retry. `**` Considered for stricter latency bounding, but rejected in favour of the simpler Feign-level connect/read timeouts: ``@TimeLimiter`` requires the wrapped method to return ``CompletableFuture``, which would have pushed the synchronous booking-creation flow onto an async model for no benefit, since Feign's own timeout already bounds worst-case latency per attempt.
5. `**Fail the booking silently / return the equipment as unavailable rather than 503 on circuit-open.**` Rejected: conflating "equipment is in use" (a real business state, ``409``) with

"we couldn't find out" (an infrastructure failure, `503`) would mislead the client into thinking the equipment itself was the problem rather than the dependency being unreachable.

Consequences

Positive

- Booking Service degrades gracefully under Equipment Service outages: callers get a fast, clear

`503` (confirmed in testing at ~1-1.3s, versus a multi-second/indefinite hang without the timeout+circuit-breaker combination) instead of a hang or raw exception.

- The circuit breaker prevents pointless repeated network attempts against a service that's known

to be down for the `wait-duration-in-open-state` window, then automatically probes recovery.

- Distinct HTTP semantics: `409` for "equipment genuinely not available" vs `503` for "couldn't verify due to a dependency failure" - a real business state is not confused with an infrastructure failure.

Negative / trade-offs

- Adds a fixed worst-case failure-path latency of up to roughly `3 attempts × ~2s timeout` before

the circuit trips open; once open, subsequent calls fail fast. This was judged an acceptable trade-off given the assignment's small scale and the value of not treating one bad probe as a full open.

- The fallback treats *any* failure the same way (503, "could not verify"), including a 404 from

Equipment Service for a genuinely non-existent equipment ID - which is also counted as a circuit breaker failure. In a production system this would likely be refined with `ignoreExceptions` so that legitimate 4xx business responses don't count against the breaker's failure budget; this was accepted as a known simplification for the assignment's scope.

- Resilience4J configuration lives in Config Server rather than code, which is consistent with the

externalised-configuration requirement but means tuning it requires a config refresh/restart rather than a code change.

Implementation Evidence

- ``booking-service/src/main/java/com/labequip/booking/service/EquipmentAvailabilityService.java``
- ``booking-service/src/main/java/com/labequip/booking/client/EquipmentClient.java``
- ``config-server/src/main/resources/config/booking-service.yml`` (``resilience4j.*``, ``feign.client.config``)
- Resilience evidence transcript: ``docs/evidence-resilience.txt`` - circuit breaker state

``CLOSED`` → repeated ``503`s` → ``OPEN`` (``failureRate: 80%``, ``notPermittedCalls: 17``) → automatic ``HALF_OPEN`` transition

- Report evidence: Section 2.3 "Service-to-Service Communication and Resilience"
- Screenshot timestamp: ``**TBD - insert after recording**``